

Deployment of the Application

Introduction

Deployment is the final stage of application development where the project is made runnable for users. In this project, deployment ensures that the AI Video Script Generator works properly in a local environment before moving to any cloud platform.

The deployment process focuses on making the Flask application stable, testable, and ready for real usage.

Preparing the Application for Local Deployment

Before running the project locally, certain steps are required to ensure smooth execution.

Step 1: Organizing Project Structure

The project must be properly structured as:

```
AI-Video-Script-Generator/  
├── app.py  
├── templates/  
│   └── index.html  
├── static/  
│   └── style.css  
├── logs.json  
├── requirements.txt  
└── prompt_logger.py
```

Step 2: Installing Dependencies

All required Python libraries must be installed.

```
pip install flask  
pip install groq
```

Optional:

```
pip freeze > requirements.txt
```

Step 3: Configuring API Key

The Groq API key must be set securely in the environment or code.

Example:

```
import os  
os.environ["GROQ_API_KEY"] = "your_api_key_here"
```

Step 4: Ensuring Backend Configuration

- Flask app should have correct routes
- API integration must be tested
- Prompt builder module should be connected
- Logging system should be active

Example run setup:

```
if __name__ == "__main__":  
    app.run(debug=True)
```

Step 5: Checking Internet Connection

Since Groq API is cloud-based:

- Stable internet connection is required
 - API calls must be tested before deployment
-

8.3 Local Testing and Verification

After setup, the application is tested locally to ensure correct functionality.

Step 1: Running the Flask Server

```
python app.py
```

The application runs on:

```
http://127.0.0.1:5000/
```

Step 2: Testing User Input

User inputs are tested with different combinations:

- Topic: AI in Education
 - Tone: Motivational
 - Platform: YouTube
 - Duration: 2 minutes
-

Step 3: Verifying AI Output

Check if:

- Script is properly structured (Intro, Body, Conclusion)
 - Tone matches input
 - Output is relevant to topic
-

Step 4: Testing Prompt Logging System

Verify:

- Prompt is stored correctly in `logs.json`
 - Response is saved with timestamp
 - No data loss occurs during multiple requests
-

Step 5: Error Handling Verification

Test scenarios:

- Empty input fields
- Invalid API response
- Slow internet connection

Ensure:

- App does not crash
 - Proper error messages are displayed
-

Step 6: UI Testing

Check frontend:

- Input form alignment
 - Button functionality (Generate / Reset)
 - Output display formatting
-

8.4 Observations from Testing

After local testing, the following observations were made:

- The system successfully generates AI-based scripts
- Response time is fast due to Groq API
- Prompt quality directly affects output quality
- Logging system works correctly for tracking inputs and outputs
- Flask backend handles requests efficiently